(6) Specification - Diff. things to diff. people
- can be done by most products

(7) Requirement Management

Ch-5 Functional Independence

Defn. A module performs a single task and needs very little interaction with other modules

→ Advantages of fun^{n} Indp^{n}

(1) Error Isolation
- It reduces the chances of the error propagating to the other modules.
- It makes it easy to locate the error.

(2) Scope of reuse
- Reuse of a module for the development of other app^{n}s becomes easier.

(3) Understandability
- When modules are fun^{c}lly independent, complexity of the design is greatly reduced

→ Classification of cohesiveness

- Cohesiveness of a module is the degree to which the diff. fun^{n}s of the module co-operate to work towards a single objective.

Low ⇒ coincidental → logical → Temporal → Procedural
High ⇒ Functional ← Sequential ← communicational ←

(1) Functional - IF 2 operations present within a module perform the same functional task OR they are part of the same "purpose"
- each element does some related task

(2) Sequential cohesion
In a module if 2 operations are such that x's output ⇒ y's input
where, x, y ∈ same module

(3) communicational
- Elements inside a module that operate on same i/p data OR contribute to same data.

(4) Procedural
- Modules whose Instructions do different tasks, but to ensure a particular order in which tasks are performed, they are put into same module.

(5) Temporal cohesion
- Instructions that must be executed during same time span are put together.

(6) Logical
- when modules have logically similar instructions/ elements.

(7) Coincidental
- Instructions are not related with each other

Ch: 5

- Classification of coupling
  ↳ degree of interdependence

Low

(1) Data – Two modules are data coupled, if they communicate using elementary data item

(2) Stamp – Two modules are stamp coupled, if they communicate using a composite data

(3) Control – If data from one module is used to direct the order of instruction execution in another.
eg flag set in one module & modified in another module

(4) Comman – If they share some global data items.

(5) Content – If two modules share code. jump from one module into the code of another module.

High

→ A layered design achieves control abstraction

Date
Page

- Terminologies associated with layered design:

(1) Superordinate & Subordinate
↓ module that controls another module
↓ module controlled by another module

(2) Visibility - module B is said to be visible to A if A directly calls B.

(3) Control abstraction - ~~functions~~ modules at the higher layers should not visible to the modules at the lower layers.

(4) Depth & width

(5) Fanout - measure of the no. of modules that are directly controlled by a given module.

(6) Fan in - no. of modules that directly invoke a given module.

Date
Page

- **Function-oriented design**

foll^{n} are the two approach:

(1) Top-down decomposition
- Starting at a high-level view of the system, each high-level function is successively refined into more detailed functions.

eg High-level function = create new library member
detailed fun^{s} = assign membership no.
= Create member record
= print bill

(2) Centralised system state
- The values of certain data items that determine the response of the system to a user action or external event.

eg set of books determines the state of a library automation system.

- **Object oriented design**

- The system state is decentralised.
- Three important concepts associated with AOT

Date
Page

① Data Abstraction.
- Principle of data abstraction implies that how data is exactly stored is abstracted away.

Eg. Consider an ADT such as a stack. stack object may internally be stored in an array, a linearly linked list, or a bidirectional ll. the external entity has no knowledge of this and can use stack only with push / pop.

② Data structure - It is constructed from a collection of primitive data items.

③ Data Type - ADTs are user defined data types.

→ Three advantages of using ADTs.

i) The data objects are encapsulated within the methods so if error occures then it will reside locally. And it will not propogate to other objects. also error can easily be traced.

ii) An ADT based design displays high cohesion and low coupling.

iii) it makes the design solution easily understandable.

• Diff bet^n function oriented design & object oriented design.

⇒ Unlike FOD methods in OOD, the basic abstraction is not the services available such as issue-book, display-book-details etc but real-world entities such as member, book is in OOD.

→ In OOD, state information exists in the form of data distributed among several objects of the system.
- In FOD, the state information is available in a centralised shared data store.

→ FO technique, group fun^s together if, as a group they constitute a higher level fun^c
- OO technique group fun^s together on the basis of the data they operate on.

⟹ Objects – nouns
function – verbs